Python Exercises

The following exercises are to be prepared by the fourth session of the course (9th of November). Until this session, mark all exercises which you are able to present as such on our e-learning platform. Also make sure that you have submitted your work packaged in a `zip` file, as described in Appendix A.
A student will be picked at random to be interviewed about their solution. If there are clear doubts that the student can explain the presented work, no exercise of this sheet will be counted towards the final grade. The solution will be further discussed with the plenum, for a common learning experience.

Unless stated otherwise, do **not** solve exercises by use of imported libraries. Specifically, you **are allowed** to use the `setuptools, csv, os, pathlib, stat, datetime` libraries for this exercise sheet.

1 Python Package

Create a python package, called `aaip` that holds all of the code for all the following exercises. In this package, each of the following exercises gets a module called `ex1`, `ex2`, `ex3` and so on.
Your package shall contain a function in the first module, called `hello_python_exercises(...)`. This function shall take the following arguments:

- an integer `num_students` and
- a string `student_name`

The function parameter `student_name` shall have a default argument, namely your personal name.
When calling the function, a message shall be printed: "This course has `<num_students>` students and one of those is `<student_name>`."
Thereby, the output depends on `num_students` and `student_name`.
To demonstrate your first package and module, create a script outside the scope of the package, import the module and call the function with different parameters. Appendix A shows how to set up, install and hand in such a package.

2 Simple File I/O

Create a module that handles a very simple database of students with several functions:

- a function that opens, reads, and returns the content of a csv file as a list. The default value for the file name is `students_list.csv`. Use the file `students_list.csv` that we provide together with this sheet: its first row is the header `id,name,grade`, and every further row contains the id, the name, and the grade of one student. The function reads every row after the header and stores each `student` in a dictionary with the keys `id`, `name`, and `grade`. That is, the function returns

```
[
 {"id": <id1>, "name": <name1>, "grade": <grade1>},
 {"id": <id2>, "name": <name2>, "grade": <grade2>},
 ...
 ]
```

- a function, that takes a database (i.e., a list) as a parameter and adds iteratively new students to that list. The function adds students to the database, as long as no *empty* student name (empty string `""`) is passed. Think about an appropriate way to handle the assignment of new identifiers.
- a search method, that searches for either identifiers `id` or student names `name` and outputs the `id`, `name`, and `grade` in a nicely formatted way.
- a method to search for a specific student and change the corresponding `grade`.
- a method to store the database back to a file. The default value for the file name is `students_list.csv` and is compatible to the input file, so this file can be read again.

Test your module by creating a script outside the scope of your package and test through all the methods.

3 Complex Number Class with Operator Overloading

Create a `ComplexNumber` class that represents a complex number (plain, do not use the data type complex; represent a complex number with two attributes, real and imaginary) and supports basic arithmetic operations (addition, subtraction, multiplication, division) through operator overloading.
Requirements: it must be possible to add, subtract, multiply, and divide complex numbers like integers: `a+b`, `a-b`, `a*b`, and `a/b`.
Extend the class in such a fashion, so when you print an instance of the class, the complex number is displayed in the form a + bi.
Write a function that takes a list of `ComplexNumber` objects and returns their sum, using the built-in function `sum()`. Note that `sum()` starts from the integer `0`, i.e., it first computes `0 + c` for the first element `c` of the list.
Hint: We have done something very similar with the class `Number` in the lecture. For further operator overloading have a look at this link from the Python Reference.

Test your module by creating a script outside the scope of your package and test through all the methods.

4 File and Directory Management

In this exercise, you will create a Python module that utilizes the `os` package and `pathlib` package extensively for file and directory management operations. Prefer `pathlib` if you can. The module will involve creating, modifying, traversing, and removing files and directories. You will also handle file metadata and permissions, and make use of the `os` package to interact with the operating system. The Task:

4.1 Directory and File Creation

Write a function `create_project_structure(base_path, project_name)` that creates a directory structure for a new project. The structure should include:

- A root folder with the project name.
- A subfolder named `src/` for source code.
- A subfolder named `docs/` for documentation.
- A file named `README.md` inside the root folder.
- A file named `main.py` inside the src/ folder.

Requirements:

- If the project folder already exists, print a message indicating that.
- Ensure that the directories are created recursively.
- Write placeholder content into `README.md` and `main.py`.

4.2 Directory Traversal

- Write a function `traverse_directory(path)` that recursively traverses the directory structure starting from path, printing out each file and folder.
- The output should clearly distinguish between files and directories.

4.3 File Metadata and Permissions

Write a function `file_metadata(file_path)` that:

- Retrieves and prints the file size, last modified time, and creation time. The creation time is not available on every operating system: use `st_birthtime` where it exists (macOS, Windows) and fall back to `st_ctime` otherwise (on Linux, this is the time of the last metadata change).
- Changes the file permissions to make it read-only, then changes it back to be writable.

Use the `stat()` method for retrieving metadata, and `chmod()` method for changing permissions.

4.4 Find Files by Extension

- Write a function `find_files_by_extension(path, extension)` that searches for all files with the specified extension (e.g., `.py`, `.md`) within the given directory (and its subdirectories).
- Return a list of full paths to the files found.
- You can use a combination out of `walk()` and `glob()` methods to make life easy.

4.5 Deleting Files and Directories

- Write a function `delete_files_and_directories(path)` that:
  - Deletes all files in the directory.
  - Deletes all subdirectories.
  - Deletes the directory itself.
- Ensure the function handles nested directories and files.
- Use `os.remove()` or pathlib's `unlink` to delete files and one of the `rmdir()` methods for directories.

Test your module by creating a script outside the scope of your package and test through all the methods.

A Structuring and Handing in Your Package

All code of this sheet lives in one self-contained package that we can install and run on our machines. This appendix shows the minimal setup we expect.

Why scripts cannot simply import your package

A statement like `import aaip` only works if Python finds `aaip` on its search path `sys.path`. When you run `python scripts/run_ex1.py`, this path contains the folder of the script (`scripts/`), the standard library and all *installed* packages, but not the sibling folder `src/`. Hence, the import fails with

```
ModuleNotFoundError: No module named 'aaip'
```

Installing your package once fixes this for every script, no matter where it is located.

Folder structure

```
aaip_<lastname>/            <- the root folder you hand in
    pyproject.toml          <- describes the package (see below)
    students_list.csv       <- provided with this sheet
    src/
        aaip/               <- the package
            __init__.py     <- marks aaip as a package, may be empty
            ex1.py          <- one module per exercise
            ex2.py
            ex3.py
            ex4.py
    scripts/                <- scripts "outside the scope of the package"
        run_ex1.py
        run_ex2.py
        ...
```

A script imports from your package like from any other library, e.g.

```
from aaip.ex1 import hello_python_exercises

hello_python_exercises(20)
```

The file `pyproject.toml`

Copy the following into the file `pyproject.toml` in the root folder. The `[build-system]` table tells the installer to build your package with `setuptools`, which finds the package in `src/` automatically. The `[project]` table names your package.

```
[build-system]
requires = ["setuptools>=64"]
build-backend = "setuptools.build_meta"

[project]
name = "aaip"
version = "0.1.0"
requires-python = ">=3.12"
```

Installing and running

Create the root folder anywhere, e.g. in the `work/` folder of the course repository (which git ignores), and run all commands from inside the root folder.

With uv: `uv run` creates a virtual environment `.venv/` for your package, installs the package into it and runs the script:

```
uv run python scripts/run_ex1.py
```

Without uv, e.g. on the university lab PCs (see the README of the course repository):

```
python -m venv .venv
.venv\Scripts\python -m pip install -e .
.venv\Scripts\python scripts\run_ex1.py
```

On macOS and Linux, replace `.venv\Scripts\python` by `.venv/bin/python` (and use `python3` in the first line if `python` is not found). The flag `-e` (*editable*) installs a link to your code instead of a copy, so changes to your modules take effect without reinstalling.

Relative file paths

A relative file name such as `students_list.csv` is looked up in the folder you run the script *from* (the current working directory), not next to the script. Hence, always run your scripts from the root folder, or build the path relative to the script with `pathlib`:

```
from pathlib import Path

csv_path = Path(__file__).parent.parent / "students_list.csv"  # scripts/ -> root folder
```

Handing in

1. Make sure that every exercise has a script in `scripts/` that runs without errors.
2. Delete the folders `.venv/`, `src/aaip.egg-info/` and all `__pycache__/` folders. The installation creates them, and we recreate them on our side.
3. Zip the root folder as `aaip_<lastname>.zip`.
4. Self-check: unzip the file into an empty folder, then repeat the installation and run all scripts there. If everything runs, it will run for us as well.